

Capstone Workflow:

Hospital Readmission Prediction

Introduction

This capstone focuses on building a complete, reproducible MLOps system that predicts whether a diabetic inpatient will be readmitted to hospital within 30 days of discharge. The emphasis is on the full operational lifecycle of a machine-learning model in production — not just training a classifier but operating it responsibly: validate → train → track → register → deploy → monitor → retrain → govern.

Learners design and implement an end-to-end workflow that begins with understanding the clinical business objective, analysing and preparing a real, messy healthcare dataset, and establishing a performance baseline with a simple model. The project then progresses through an advanced model, full experiment tracking and a model registry, productionisation (API + container + CI), and finally a monitoring-and-retraining loop driven by data and prediction drift.

The two progressively enhanced models are:

- **Baseline model:** Logistic Regression (interpretable reference point) — Solution V1
- **Advanced model:** XGBoost with imbalance handling — Solution V2

Both models are tracked in MLflow, the best is registered and promoted to a production alias, served through a FastAPI endpoint, containerised with Docker, validated by GitHub Actions CI, and continuously monitored for drift so it can be retrained when its inputs or outputs shift.

Business Problem

30-day hospital readmissions are costly and often preventable. If a hospital can flag high-risk patients at the point of discharge, care teams can intervene with follow-up calls, medication review and patient education, reducing avoidable readmissions and the associated penalties and bed pressure.

A predictive model is, however, only useful in production if it is reproducible, deployable, monitored and maintainable. A model that scores well in a notebook but cannot be served, versioned, watched for drift, or safely retrained delivers no clinical value. This capstone therefore frames the problem as an MLOps problem: produce a model and the operational machinery that lets a hospital trust, serve and maintain it over time.

The core difficulty is twofold. First, the data is messy and leakage-prone — multiple encounters per patient, missing values, near-empty columns, and outcome codes (death/hospice) that invalidate the label. Second, the target is highly imbalanced (~9% positive), so naive accuracy is

misleading and the costly error — a missed readmission (false negative) — must be controlled deliberately through metric choice and model selection.

Data Description

Diabetes 130-US Hospitals (1999-2008): A public, de-identified dataset of inpatient encounters for diabetic patients across 130 US hospitals.

1. Structure and Target

- **101,766 encounters × 50 features:** One row per hospital admission for a diabetic patient.
- **Target:** Derived from readmitted → 1 if `<30` days, else 0 (no leakage from the >30/NO classes is introduced — the binary task is "readmitted within 30 days or not").
- **Granularity caveat:** A single patient (patient_nbr) can appear in many encounters, so rows are not independent — a critical leakage risk addressed in Stage 2.

2. Feature Groups

- **Demographics:** Age (binned), race, gender.
- **Admission/discharge context:** Admission type/source, discharge disposition, time in hospital.
- **Clinical activity:** Number of lab procedures, procedures, medications, outpatient/emergency/inpatient visits, number of diagnoses.
- **Diagnoses:** ICD-9 codes in diag_1, diag_2, diag_3 (high-cardinality).
- **Medications:** 20+ drug columns (e.g. metformin, insulin) recording change status.
- **Labs:** A1Cresult, max_glu_serum.

3. Data Quality and Constraints (assess before modelling)

- **Missing values:** Are encoded as ? and must be loaded as NaN. Representative missingness: weight ~97% (drop), medical_specialty ~49%, payer_code ~40%, race ~2%.
- **Patient-level leakage:** Keep the first encounter per patient so train/test rows are independent — duplicate patients across splits would leak.
- **Label validity:** Drop expired/hospice discharges (discharge_disposition_id ∈ {11,13,14,19,20,21}) — those patients cannot be readmitted.
- **Class imbalance:** After cleaning, ~69,973 rows at ~9% positive (representative) — the modelling frame is imbalanced and metric choice must reflect this.
- **High cardinality:** ICD-9 diagnoses and medical_specialty must be grouped before encoding.
- **Constant/zero-variance** medication columns carry no signal and should be dropped.

Overall Methodology and Learning Outcomes

The project follows a structured MLOps lifecycle emphasising reproducibility, leakage-safety, objective evaluation and evidence-based operation, organised into four stages delivered as four weekly milestones.

Stage 1 — Business Objective, Data Understanding & Methodology

- Define the clinical objective and measurable success criteria
- Inspect the dataset (shape, target derivation, patients vs encounters)
- Perform univariate, bivariate and missing-value EDA with interpretation

and design a leakage-safe preprocessing and end-to-end MLOps architecture.

Stage 2 — Data Preparation

- **Clean the data:** ? → NaN, first-encounter deduplication, and removal of expired/hospice records and unusable columns
- **Engineer dataset-specific features:** ICD-9 grouping, age midpoint, service utilisation, medication-change count
- **Transform features inside a single ColumnTransformer:** Create the train/validation/test splits first, then fit preprocessing on the training split only.

Stage 3 — Model Development & Experiment Tracking

- Train a Logistic-Regression baseline and an XGBoost advanced model as full sklearn pipelines with imbalance handling
- Evaluate with imbalance-aware metrics
- Log both runs to MLflow
- Register the best model and promote it to the production alias.

Stage 4 — Productionisation & Operations

- Serve the registered model with FastAPI
- Containerise with Docker
- Automate tests with GitHub Actions
- Monitor data + prediction drift with Evidently + PSI
- Define and run a multi-signal retraining workflow and add prediction logging and model governance — capturing all operational evidence inside the submitted notebook and report.

Learning Outcomes

By completing this capstone you will be able to:

1. Assess data quality and leakage risks on a real, messy clinical dataset.
2. Design leakage-safe preprocessing pipelines (split first; fit preprocessing on train only; deduplicate at the patient level).

3. Engineer and justify dataset-specific features and encode/scale them appropriately.
4. Train and fairly evaluate classifiers on an imbalanced target, selecting problem-appropriate evaluation metrics (ROC-AUC / PR-AUC / recall — see Evaluation Metrics).
5. Track experiments and manage model versions with MLflow (registry + promotion).
6. Serve a model with FastAPI, containerise it with Docker, and automate validation with GitHub Actions.
7. Detect data and prediction drift and design a multi-signal retraining workflow.
8. Apply logging and governance for a maintainable, reproducible system, and capture operational evidence inside submittable artefacts (notebooks/report).

Evaluation Metrics

The headline metrics are ROC-AUC (model selection) and recall (operational watch), recommended because of the characteristics of this dataset and problem:

- **ROC-AUC / PR-AUC:** Threshold-independent ranking quality on an imbalanced target.
- **Recall:** The rare positive class (readmission) is the costly one to miss; recall watches false negatives directly.
- **Precision / F1:** Reported alongside to expose the precision/recall trade-off.
- **Accuracy is explicitly NOT used for selection:** A "never readmitted" model scores ~91% and is useless. Justify your metric choice from the problem, not by habit for a different cost structure, a different metric set would be appropriate.

Project Stages

Stage 1 — Business & Data Understanding, EDA, Methodology [20 Marks]

Task 1.1 — Business Understanding [5 Marks]

- **1.1.1 — Business objective & justification [2]:** State a clear, specific objective (reduce avoidable 30-day readmissions) and justify it (cost/clinical impact, who acts on the prediction).
- **1.1.2 — Measurable success criteria & metric choice [2]:** Define measurable targets with a problem-appropriate metric (ROC-AUC/recall), tied to the ~9% imbalance and the cost of missed readmission.
- **1.1.3 — Scope, constraints & assumptions [1]:** State scope, constraints and assumptions (local execution, file-store MLflow, imbalanced target).

Task 1.2 — Dataset Understanding [4 Marks]

- **1.2.1 — Dataset structure, target derivation & patients-vs-encounters [2]:** Print the shape, derive the target ($<30 \rightarrow 1$), and distinguish patients from encounters.

- **1.2.2 — Feature types & data-quality / missing-value quantification [2]:** Summarise feature types and quantify missingness per column (e.g. weight ~97%).

Task 1.3 — Exploratory Data Analysis [7 Marks]

- **1.3.1 — Univariate analysis [3]:** Visualise the target imbalance and ≥ 3 numeric distributions with observations.
- **1.3.2 — Bivariate analysis vs target [2]:** Analyse ≥ 1 relationship to the target (e.g. readmission rate vs prior inpatient visits) with interpretation.
- **1.3.3 — Visualisations + written interpretation [2]:** Clear, labelled plots, each key finding interpreted in writing.

Task 1.4 — Methodology Design [4 Marks]

- **1.4.1 — End-to-end MLOps workflow / architecture [2]:** Describe the workflow: validate $\rightarrow$ train $\rightarrow$ track $\rightarrow$ register $\rightarrow$ deploy $\rightarrow$ monitor $\rightarrow$ retrain $\rightarrow$ govern.
- **1.4.2 — Data-leakage-prevention strategy justified [2]:** Justify split-first, fit-on-train-only, first-encounter deduplication.

Stage 2 — Data Preparation [25 Marks]

Task 2.1 — Data Cleaning [6 Marks]

- **2.1.1 — Missing value handling & invalid value treatment [2]:** ? $\rightarrow$ NaN; drop weight and constant/zero-variance columns.
- **2.1.2 — Patient deduplication & removal of invalid records [2]:** Keep the first encounter per patient; remove expired/hospice discharges.
- **2.1.3 — Target variable creation & validation [2]:** Build the binary target and validate its class balance (~9% positive over ~69,973 rows).

Task 2.2 — Feature Engineering [8 Marks]

- **2.2.1 — ICD-9 diagnosis grouping into clinical categories [4]:** Group diag_1/2/3 into clinical categories with a justified mapping.
- **2.2.2 — Additional engineered features, justified [4]:** At least three of the following: age midpoint, service utilisation, medication-change count, specialty top-k grouping; each justified.

Task 2.3 — Feature Transformation [5 Marks]

- **2.3.1 — Numeric scaling & categorical encoding [3]:** Median-impute + scale numerics; impute + one-hot encode categoricals (handle_unknown='ignore').
- **2.3.2 — Leakage-safe ColumnTransformer/Pipeline assembly [2]:** Assemble one transformer that can be fit on train only.

Task 2.4 — Validation & Splitting [6 Marks]

- **2.4.1 — Stratified train/validation/test split created first [3]:** Stratified split before any fitting (`random_state=42`, ratio preserved).
- **2.4.2 — Leakage-safe fit-on-train + class-balance check [3]:** Fit preprocessing on the training split only and confirm that class balance is preserved across splits.

Stage 3 — Model Development & Experiment Tracking [30 Marks]**Task 3.1 — Baseline Model [5 Marks]**

- 3.1.1 — Logistic Regression baseline trained (Pipeline, imbalance handled) [3]
- 3.1.2 — Baseline evaluated on validation [2]

Task 3.2 — Advanced Model [8 Marks]

- 3.2.1 — XGBoost trained (Pipeline, `scale_pos_weight`` imbalance handled) [5]
- 3.2.2 — Advanced model evaluated on validation [3]

Task 3.3 — Model Evaluation [5 Marks]

- **3.3.1 — Imbalance-aware metrics + model selection [3]** — ROC-AUC, PR-AUC, recall, precision, and F1; select the final model using ROC-AUC.
- **3.3.2 — Plots + interpretation (why accuracy misleads) [2]** — Confusion + ROC plots with interpretation.

Task 3.4 — MLflow Experiment Tracking [7 Marks]

- 3.4.1 — Both runs logged with params + metrics + model [4]
- 3.4.2 — Runs compared (`search_runs` table) [3]

Task 3.5 — Model Registry & Versioning [5 Marks]

- 3.5.1 — Best model registered in the MLflow Model Registry [2]
- 3.5.2 — Promotion to production alias + version history [3]

Stage 4 — Productionisation & Operations [25 Marks]**Task 4.1 — API Deployment [5 Marks]**

- 4.1.1 — FastAPI inference service (`/predict` + `/health` + validation) [3]
- 4.1.2 — Live request/response demonstrated in-notebook [2]

Task 4.2 — Containerisation [4 Marks]

- 4.2.1 — Valid Dockerfile [2]
- 4.2.2 — Build/run evidence [2]

Task 4.3 — CI/CD Automation [5 Marks]

- 4.3.1 — GitHub Actions workflow (install deps + run pytest) [3]
- 4.3.2 — Passing-test evidence [2]

Task 4.4 — Monitoring & Drift Detection [4 Marks]

- 4.4.1 — Feature drift (Evidently, reference vs current) [2]
- 4.4.2 — Prediction drift (PSI on scores) + interpretation [2]

Task 4.5 — Retraining Workflow [3 Marks]

- **4.5.1 — Multi-signal retraining trigger defined [1]:** Prediction PSI > 0.2 OR drift share > 0.3 OR dataset drift.
- 4.5.2 — Retraining workflow runs + re-registers + decision record [2]

Task 4.6 — Logging & Governance [4 Marks]

- 4.6.1 — Prediction logging [2]
- 4.6.2 — Model governance (registry history + alias + pinned deps) [2]

Deliverables

#	Deliverable	Format	Covers
1	Hospital_Readmission_MLOps_Report	PDF	Stages 1, 3, 4 — business, methodology, results, monitoring, governance, and Stage-4 operational evidence (Docker/CI/MLflow/drift/logging screenshots + summaries)
2	Data_Preparation.ipynb	Executed notebook	Stage 1 (EDA) + Stage 2 (preparation)
3	Model_Development_and_Tracking.ipynb	Executed notebook	Stage 3 (modelling + MLflow tracking + registry)
4	Operations_Monitoring_and_Evidence.ipynb	Executed notebook	Stage 4 — API demo, container & CI evidence, drift monitoring, retraining, logging/governance
5	app.py	Python script	Stage 4 FastAPI inference service source
6	data_prep.py	Python script	Data loading, preprocessing, feature engineering, train-test split, and data validation
7	train.py	Python script	Model training, experiment tracking (MLflow), hyperparameter logging, and

			model registration
8	evaluate.py	Python script	Model evaluation, performance metrics, validation, and model selection
9	retrain.py	Python script	Automated retraining workflow, model update, and registry integration
10	monitoring.py	Python script	Drift detection, performance monitoring, logging, Evidently reports, and governance monitoring

Note: Model artefacts (.pkl/.bin, MLflow store, Docker images, drift HTML) are NOT required. Evidence is captured through notebook implementation, in-notebook outputs, listings, screenshots, training/prediction logs, and reproducibility documentation in the report.

Tools, Libraries, and Technology Stack

This capstone uses an end-to-end MLOps technology stack that spans data preparation, model development, experiment tracking, deployment, monitoring, and retraining workflows.

Programming Environment

- Python 3.11+
- Jupyter Notebook or JupyterLab
- VS Code (optional)

Data Processing and Analysis

- pandas
- numpy
- scikit-learn
- matplotlib
- seaborn

Model Development

- Logistic Regression (Baseline Model)
- XGBoost (Advanced Model)
- imbalanced-learn (optional)

Experiment Tracking and Model Registry

- MLflow Tracking Server

- MLflow Model Registry

API Development

- FastAPI
- Uvicorn

Containerisation

- Docker
- Docker Compose (optional)

CI/CD and Testing

- GitHub Actions
- pytest

Monitoring and Drift Detection

- Evidently
- Population Stability Index (PSI)

Supporting Libraries

- joblib
- pydantic
- pathlib
- logging
- json
- yaml (optional)

Implementation Guidance

Learners are advised to follow the implementation sequence below.

1. Complete data understanding and exploratory analysis before implementing any cleaning or modelling steps.
2. Create the train, validation, and test splits before fitting any preprocessing transformations.
3. Maintain reproducible experiments throughout the project:
 - `random_state = 42`

- fixed dataset partitions
 - documented hyperparameters
 - version-controlled artefacts
4. Implement all preprocessing operations inside a single leakage-safe Pipeline and ColumnTransformer.
 5. Establish the Logistic Regression baseline before implementing the XGBoost model.
 6. Log both models completely in MLflow:
 - Parameters
 - Metrics
 - Model artefacts
 - Run metadata
 7. Register and promote only the selected model version to the production alias.
 8. Validate each operational component independently before integration:
 - FastAPI service
 - Docker container
 - GitHub Actions workflow
 - Drift detection pipeline
 - Retraining workflow
 9. Capture operational evidence throughout implementation, including:
 - MLflow screenshots
 - API request and response examples
 - Docker build evidence
 - CI execution evidence
 - Drift reports
 - Prediction logs
 - Retraining decisions

The emphasis of this capstone is on building a reproducible and maintainable machine learning system rather than simply achieving the highest predictive performance.

Common Mistakes and Troubleshooting

Data Understanding and Preparation

- Treating patient encounters as independent observations.

- Failing to remove duplicate patients before splitting.
- Retaining expired or hospice discharge records.
- Loading '?' values as strings instead of missing values.
- Fitting preprocessing transformations before creating dataset splits.

Feature Engineering

- Applying inconsistent ICD-9 grouping rules.
- Creating features without documenting their clinical justification.
- Introducing data leakage during feature engineering.

Model Development

- Selecting models based solely on accuracy.
- Ignoring class imbalance during model training.
- Comparing models using different dataset partitions.
- Evaluating directly on the test set during development.

Experiment Tracking

- Logging metrics without parameters.
- Failing to register the final model version.
- Overwriting previous experiments.
- Not recording model versions and aliases.

Deployment and Operations

- Loading preprocessing separately from the trained Pipeline.
- Exposing prediction endpoints without input validation.
- Building Docker images that do not reproduce the local environment.
- Creating CI workflows that do not execute automated tests.

Monitoring and Retraining

- Monitoring only feature drift and ignoring prediction drift.
- Using PSI values without interpretation.
- Defining retraining using a single signal.
- Retraining models without recording decisions and evidence.

General Recommendation

Validate each stage independently before progressing to the next stage. Preserve intermediate artefacts, maintain experiment records, and capture operational evidence throughout implementation.

Appendix 1 — Monitoring & Retraining (operational design)

Monitoring and retraining are assessed on operational clarity, not just on running a drift tool.

- **Reference dataset:** At training time, a sample of the training features is saved (reference_sample.csv) — the fixed baseline distribution every future batch is compared against.
- **Monitoring cadence:** In production each incoming batch of new encounters is scored on a regular cadence; here a simulated "current" batch with realistic shifts (longer stays, more medications, older skew, more circulatory diagnoses) stands in for live data.
- **Reports generated:** Evidently DataDriftPreset → feature-drift report (drift_report.html), and a prediction PSI on model output scores → machine-readable summary (drift_summary.json). Both are summarised and screenshotted inside the operations notebook/report (the raw HTML/JSON are not uploaded).
- **Multi-signal retraining trigger (not PSI alone):** Retrain if prediction PSI > 0.20 or drifted-feature share > 0.30 or Evidently flags dataset drift. In the reference run only the prediction-PSI trigger fires — a deliberate teaching point that prediction drift can be significant even when feature-level dataset drift reads False.

Note: Thresholds are representative examples for instruction. PSI > 0.20, drift share > 0.30 and the 0.50 classification threshold are sensible defaults for this exercise; in a real deployment they would be tuned to the hospital's tolerance for false negatives vs. alert fatigue.

Appendix 2 — Assumptions & Constraints

- **Local execution:** MLflow uses a local file-store tracking server + registry.
- **The target is imbalanced (~9%) → select models using ROC-AUC and monitor recall:** Accuracy is misleading.
- **Splits are created first; all preprocessing is fit on the training split only:** (inside the model Pipeline) and applied to validation/test — no test information leaks into training.
- **Reproducible (random_state=42) and idempotent:** Outputs saved to files and evidenced in the notebooks.

Appendix 3 — Glossary

- **Leakage prevention:** Never letting test information influence training (split first, fit preprocessor on train, deduplication patients).
- **MLflow registry / alias:** Versioned model store the production alias marks the served model.
- **Drift:** Distribution shift after deployment, including feature drift (inputs), dataset drift (Evidently), and prediction drift (model outputs via PSI).
- **PSI:** Population Stability Index > 0.2 indicates a significant shift (representative threshold).

- **Retraining trigger:** A measurable condition (prediction PSI, drift share, or dataset drift) that launches a retrain + re-register + promote/rollback.

Disclaimer: All content and material on the upGrad website is copyrighted material, belonging to either upGrad or its bonafide contributors, and is purely for the dissemination of education. You are permitted to access, print, and download extracts from this site purely for your own education only and on the following basis:

- You can download this document from the website for self-use only.
- Any copies of this document, in part or full, saved to disc or to any other storage medium, may be used for subsequent, self-viewing purposes or to print an individual extract or copy for noncommercial personal use only.
- Any further dissemination, distribution, reproduction, or copying of the content of the document herein, or the uploading thereof on other websites, or use of the content for any other commercial/unauthorized purposes in any way that could infringe the intellectual property rights of upGrad or its contributors, is strictly prohibited.
- No graphics, images, or photographs from any accompanying text in this document will be used separately for unauthorized purposes.
- No material in this document will be modified, adapted, or altered in any way.
- No part of this document or upGrad content may be reproduced or stored on any other website or included in any public or private electronic retrieval system or service without upGrad's prior written permission.
- Any rights not expressly granted in these terms are reserved.